

UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE ED ELETTRICA E
MATEMATICA APPLICATA



Tesi Di Laurea in
INGEGNERIA INFORMATICA

Titolo

Edugames: App di apprendimento Infantile Interattivo

Relatore:

**Chiarissimo Prof.ssa
Carmen De Maio**

Candidato:

**Salvatore Bruno
Mat. 0612705112**

ANNO ACCADEMICO 2022/2023

Dedicata a mio nonno Salvatore e ai miei genitori, fonti di ispirazione e sacrificio.

Introduzione	4
1 Panoramica delle tecnologie utilizzate	8
1.1 MacBook Pro M1(2020)	9
1.2 iPhone 13 (2021)	10
1.3 Panoramica generale di iOS	11
1.3.1 Swift	12
1.3.2 SwiftUI	13
1.3.3 UIKit	13
1.3.4 GameplayKit	13
1.4 XCode	14
1.5 Sketch UI design	15
1.6 Photoshop	15
2 Progettazione della soluzione	18
2.1 Modelli di progettazione software	18
2.2 CBL	19
2.3 Big Idea	20
2.4 Essential Question	20
2.5 The Challenge	21
2.6 Guiding Questions	21
2.7 Guiding Activities	22
2.8 Guiding Resources	23
2.9 Synthesis	24
2.10 Soluzione	24
3 Implementazione e descrizione delle funzionalità	26
3.1 Fasi di implementazione e Metodologie di approccio	26
3.2 Interfaccia	27
3.3 Divisione in Classi	28
3.4 ContentView	28

3.5	HorizontalDatePicker	30
3.6	QuizView	31
3.7	ProgressBar	31
3.8	GifImage	32
3.9	QuizHomePageView	33
3.9.1	Quiz	34
3.10	RexyGame	35
4	Conclusioni e prospettive future	38
	Bibliografia e Sitografia	40
	Ringraziamenti	42

Nell'era digitale in cui viviamo, l'utilizzo degli smartphone e di altri dispositivi mobili è diventato così comune che sembra quasi banale avere accesso immediato a una vasta gamma di servizi con un semplice tocco. Questa diffusione massiva delle tecnologie mobili ha aperto le porte a nuove prospettive e opportunità nel campo dello sviluppo di applicazioni, dando vita a una vera e propria rivoluzione nel settore.

In particolare, il concetto di sviluppo di applicazioni, o "app developing", è diventato una disciplina chiave nell'ambito dell'informatica e dell'ingegneria del software. Gli sviluppatori di app sono responsabili della creazione, del design e dell'implementazione di applicazioni per dispositivi mobili, che vanno dai semplici giochi alle complesse applicazioni per il business.

Il termine "App developing" è comunemente utilizzato per riferirsi allo sviluppo di applicazioni per dispositivi mobili, come smartphone e tablet. Rientra nell'ambito dell'ingegneria del software e coinvolge la progettazione, lo sviluppo e la distribuzione di applicazioni per piattaforme specifiche, come iOS per dispositivi Apple o Android per dispositivi basati su sistema operativo Android.

Gli sviluppatori di app utilizzano linguaggi di programmazione come Swift o Objective-C per iOS e Java o Kotlin per Android, insieme a strumenti di sviluppo come gli IDE (Integrated Development Environment) specifici per la piattaforma di destinazione.

Il procedimento generale per lo sviluppo non risulta essere ovviamente semplice e richiede un tempo e costi hardware e software che risultano essere spesso anche molto onerosi.

Oltre alla programmazione, gli sviluppatori di app devono considerare l'interfaccia utente (UI) e l'esperienza utente (UX) per creare un'applicazione intuitiva e facile da usare. Ciò richiede la progettazione di schermate, icone, navigazione e interazioni che soddisfino le aspettative degli utenti e offrano un'esperienza piacevole.

Una volta sviluppata, l'app deve essere testata su diversi dispositivi per garantire che funzioni correttamente su varie dimensioni dello schermo, versioni del sistema operativo

e configurazioni hardware. I test consentono di identificare bug, problemi di prestazioni o problemi di compatibilità che devono essere risolti prima del rilascio dell'applicazione. Infine, gli sviluppatori distribuiscono le app sugli app store, come l'App Store di Apple o il Google Play Store, dove gli utenti possono scaricarle e installarle sui propri dispositivi. Monitorano anche le prestazioni dell'app, raccolgono feedback dagli utenti e rilasciano aggiornamenti regolari per migliorare e aggiungere nuove funzionalità all'applicazione.

Risulta dunque essere chiaro che il contesto di sviluppo di un'applicazione richiede tanti passi da portare avanti e nel caso ci fosse bisogno, essere in grado di ripercorrerli per migliorare o correggere problemi. Risulta infatti utile avere conoscenze sul modello a cascata (Waterfall) ampiamente discusso da *Sommerville* [16] nel libro di ingegneria del software e studiato nel corso di *Sistemi Informativi*.



Figura 1: Schema Waterfall di progettazione di un software

Nella fattispecie, l'intero contesto precedentemente descritto risulta essere pienamente compatibile con il progetto da me portato avanti, il cui scopo risiede nello sviluppo di un'applicazione per l'apprendimento infantile.

L'obiettivo principale dell'applicazione è quello di offrire ai bambini un ambiente virtuale sicuro e interattivo, in cui possono esplorare vari argomenti, come l'alfabeto, i numeri, i colori e le forme, oltre ad offrire una serie di attività interattive, come quiz, puzzle e giochi.

EduGames è un'applicazione Educational and Entertainment specializzata per bambini di età non troppo avanzata. La versione attuale presenta due view, in ognuna delle quali sono inclusi dei quiz e dei minigiochi. In particolare il mio compito nello sviluppo



Figura 2: Schermata principale dell'app EduGames

è partito dal disegnare l'interfaccia dell'intera applicazione mediante un software apposito (*Sketch*, il quale verrà discusso nel Capitolo 1), oltre alla realizzazione della view principale (Con relativi controlli per ognuna delle sue funzioni) e del minigame "Gioca con Remy" (per la cui realizzazione è stato necessario sfruttare software terzi).

Nella prima view l'utente può avere sempre a disposizione un calendario (*Scorrevole ed automatico in base al giorno in cui si utilizza l'applicazione*) che permette di visualizzare i punteggi che ha acquisito giorno dopo giorno, in modo da tenere traccia del progresso fatto, due quiz con domande casuali che possono essere tenute in costante aggiornamento e che presentano un grado di difficoltà diverso a seconda della sezione che si sceglie. La seconda view invece oltre a permettere al bambino di acquisire conoscenze negli ambiti di studio principali (inglese, matematica, italiano), garantisce a questo la possibilità di svagarsi tramite dei minigiochi.

L'Applicazione iOS Edugames è stata progettata a partire da un'idea successiva al percorso di Apple Foundation Program, seguito dal 30 gennaio al 24 febbraio 2023 ed è stata realizzata in collaborazione con Maurizio Esposito seguendo un progetto iniziato a Marzo e terminato a fine Maggio come attività di tirocinio.

Per la realizzazione sono state seguiti tutti i passi necessari per lo sviluppo software,

a cui si sono aggiunte tutte le specifiche che un'App iOS deve rispettare in termini di UI e UX [13]. Sono stati utilizzate inoltre numerose tecnologie, sia hardware che software, come per esempio per la parte di front-end *Swift* (*proprietario Apple*) che offre un'ampia gamma di librerie e framework per creare interfacce utente interattive e responsive. Sono state utilizzate le best practice per la progettazione dell'UI, adottando gli elementi grafici e i pattern di interazione tipici di iOS. Tali tecnologie saranno trattate in dettaglio nel **Capitolo 1**

All'interno dell'app sono frequenti scambi di dati tra le interfacce per permettere il salvataggio del punteggio sia giornaliero che totale. Saranno discusse le migliori pratiche per la gestione di elementi come immagini e barre di progresso (altra classe da me progettata che sfrutta il numero di domande per sezione per avanzare in proporzione). Si esploreranno le opzioni per ottimizzare il carico e la visualizzazione delle immagini, ad esempio utilizzando la caching o il caricamento progressivo. Saranno considerate anche le tecniche per l'aggiornamento e l'animazione delle barre di progresso, al fine di offrire un feedback visivo efficace all'utente.

Lo scopo di questa tesi è dunque quello di progettare un'applicazione che assicura al bambino un'esperienza di fruizione fluida e ottimale dei contenuti educativi, garantendo contemporaneamente un'operatività corretta dei processi in esecuzione.

Durante lo sviluppo dell'applicazione, saranno adottate soluzioni progettuali e implementative atte a garantire un'interazione corretta con l'interfaccia utente, consentendo al bambino di accedere facilmente ai contenuti educativi desiderati. Saranno applicate tecniche di ottimizzazione e gestione efficiente delle risorse, al fine di evitare ritardi o interruzioni durante l'esecuzione dei processi all'interno dell'applicazione.

Saranno discusse le scelte attuate nel **Capitolo 2** e la relativa implementazione nel **Capitolo 3**.

Le conclusioni dell'elaborato verranno discusse infine nel **Capitolo 4** in cui sono riasunte le scelte di progettazione e implementazione chiave e verrà valutata l'efficacia dell'applicazione nel garantire una fruizione fluida e corretta dei contenuti educativi per i bambini.

CAPITOLO 1

PANORAMICA DELLE TECNOLOGIE UTILIZZATE

Durante il percorso di formazione all'Apple foundation numerose sono state le tecnologie che sono state utili ai fini dello sviluppo software che è seguito. In linea generale le tecnologie che sono state necessarie hanno coinvolto sia componenti hardware che software.

- **Hardware**

- **MacBook Pro (2020) con processore Apple Silicon M1** [12]: fondamentale per lo sviluppo di software su piattaforma Apple. Il processore Apple Silicon M1 offre prestazioni elevate e un'efficienza energetica notevole.
- **iPhone 13 (2021)** [11]: Fornito durante il percorso di formazione, l'iPhone 13 ha svolto un ruolo fondamentale nel test e nell'ottimizzazione delle applicazioni sviluppate per il sistema operativo iOS. Inoltre, in determinate occasioni, è stato utilizzato come videocamera per registrare alcune attività sempre atte al successivo sviluppo.
- **Connessione di rete di tipo FTTH ed FTTC**: Una connessione Internet veloce e affidabile è stata essenziale per il download di strumenti di sviluppo, l'accesso a risorse online e la collaborazione con altri componenti del team.

- **Software**

- **iOS**: Durante il percorso di formazione, ho avuto l'opportunità di lavorare con il sistema operativo iOS, imparando a sviluppare app per dispositivi come iPhone e iPad.
- **Xcode**: Xcode è il framework di sviluppo ufficiale di Apple, che fornisce un ambiente di sviluppo integrato (IDE) completo per creare applicazioni per macOS, iOS, watchOS e tvOS.

- **Sketch:** Software ampiamente utilizzato per la progettazione grafica di interfacce utente. È uno strumento potente per creare prototipi e visualizzare il design delle applicazioni.
- **Photoshop:** Photoshop è un software di creazione ed elaborazione di immagini. Sebbene non sia specifico per lo sviluppo di software, è stato utile per la creazione e la modifica di grafiche e asset utilizzati nelle applicazioni.

1.1 MacBook Pro M1(2020)

Il MacBook Pro versione 2020 che ci è stato fornito è uno dei prodotti di punta dell'Azienda di Cupertino, in particolare con esso è stato introdotto il processore Apple Silicon M1, che rappresenta una significativa innovazione rispetto ai precedenti Intel utilizzati. Il MacBook Pro conserva il design elegante e robusto ormai tipico dei MacBook, equipaggiato da un display retina con una risoluzione 2560x1600 pixel, supporta la tecnologia True Tone, la quale si occupa di adattare la temperatura di colore alle condizioni ambientali per un'esperienza più naturale possibile.



Figura 1.1: MacBook Pro M1 (2020)

La vera **killer feature** che contraddistingue questo dispositivo e che garantisce una grande efficienza nello sviluppo, è il suo processore. L'Apple Silicon M1 si basa su un'architettura ARM progettata appositamente per i dispositivi Apple che combina CPU, GPU e Neural Engine in un singolo Chip (*Si può infatti definire come un SOC*) garantendo dunque delle grandi prestazioni associate ad un'ottima efficienza energetica che per un portatile con una tale minimalità non sono scontate.

La **CPU** effettiva è suddivisa in due cluster da **4 core** ciascuno, ed in particolare sono suddivisi in:

- **4 Efficiency Core** sempre attivi per i compiti banali, e garantiscono un consumo energetico molto basso

- **4 Performance Core** che possono lavorare con i precedenti per aumentare le prestazioni nel caso di lavori più complessi

La **GPU** invece è costituita da **8 core** operativi su 8 pipelines ad una frequenza di 1,278 GHz, inoltre contiene 128 EU (Unità di Esecuzione) capaci di oltre 25.000 threads simultanei, con un bus a 128bit, 1024 ALU (Arithmetic Logic Unit), 64 Texture Units e 32 ROP (Raster Operation Pipeline).

Il componente **Apple Neural Engine** infine è un tipo di processore NPU (Neural Processing Unit) composto da: **16 Core** dedicati al Neural Engine: Svolgono lavori particolari quali apprendimento automatico e Machine learning (ML) alla velocità di 11 trilioni (11.000 miliardi) di operazioni al secondo

Uno schema dell'architettura del processore è di seguito riportata [1]:

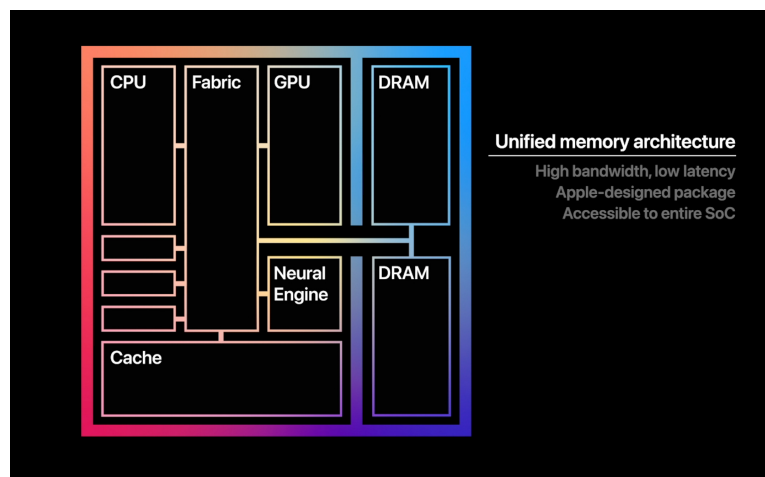


Figura 1.2: Schema architetturale Apple Silicon M1

1.2 iPhone 13 (2021)

L'iPhone 13 [11], lanciato nel 2021 come uno dei prodotti di punta dell'azienda di Cupertino (*disponibile in 4 modelli a seconda delle esigenze*), mi è stato fornito come ulteriore strumento atto allo sviluppo software. In particolare è stato utile come strumento di testing, debugging e di registrazione per alcune attività formative

Il design dell'iPhone è quello più classico oggi conosciuto, esso è dotato di un display OLED super Retina XDR, per uno schermo che in questo caso raggiunge i 6.1" di diagonale. Il tutto è alimentato dal chip proprietario "Apple A15 Bionic", il quale risulta garantire una notevole potenza ed efficienza, che consentono all'utente di eseguire qualsiasi tipo di operazione.



Figura 1.3: iPhone 13 Product Red

L'iPhone presenta una sistema di fotocamere abbastanza avanzato costituito da 2 fotocamere posteriori oltre che a quella anteriore, che permettono di registrare ottimi video e scattare foto di buon livello.

Il sistema di connessione supporta il 5G. Inoltre, il supporto al Wi-Fi 6 garantisce una connessione rapida e stabile alle reti wireless consentendo agli utenti di sfruttare al massimo le possibilità di connessione e godere di un'esperienza di navigazione senza interruzioni. Infine, il sistema operativo **iOS** gestisce tutte le funzionalità e le applicazioni dell'iPhone.

1.3 Panoramica generale di iOS

L'industria dello sviluppo delle applicazioni mobili ha raggiunto un'enorme popolarità grazie alla crescente diffusione dei dispositivi iOS oltre che alle sue dirette concorrenti. Lo sviluppo di un'applicazione iOS richiede una comprensione approfondita delle tecnologie hardware e software utilizzate nel processo di creazione. In particolare il Sistema Operativo iOS sviluppato esclusivamente per dispositivi prodotti da Apple come iPhone, iPad (*Successivamente ridenominato iPadOS nello specifico*) risulta essere uno dei sistemi più affidabili e potenti mai sviluppato, guadagnando una grande popolarità nell'ambito dei dispositivi mobili.

Le applicazioni iOS [17], possono ovviamente funzionare solo su questo tipo di sistema operativo e possono essere installate dall'App Store (piattaforma ufficiale di distribuzione Apple).

Per lo sviluppo di applicazioni native per questo sistema operativo numerosi sono i frameworks messi a disposizione da parte dell'azienda di Cupertino (alcuni dei quali proposti anche durante il corso).

Di seguito elencati alcuni tra i più importanti framework messi a disposizione (alcuni di questi utilizzati per l'effettivo sviluppo di EduGames) oltre al linguaggio di programmazione.

- **Swift:** Linguaggio di programmazione principale per app developing sotto iOS. Fornisce numerose funzionalità ed interoperabilità con i framework esistenti
- **UIKit:** Framework fron-end scritto in Objective-C per l'interfaccia grafica dell'utente (GUI) dei sistemi operativi Apple iOS e tvOS.
- **CoreData:** Framework utilizzato per la gestione e la persistenza dei dati da parte degli sviluppatori.
- **CoreGraphics:** Framework che offre supporto per il disegno all'interno dell'applicazione, ed in particolare fornisce la possibilità di rappresentare figure geometriche come linee, curve, rettangoli.
- **HealthKit:** Framework per l'integrazione di dati relativi alla salute ed al fitness all'interno delle applicazioni; consente agli sviluppatori di avere accesso ai dati dell'utente relativi all'attività fisica.
- **MapKit:** Framework per l'integrazione delle mappe nelle app iOS.
- **SiriKit:** Framework che permette alle applicazioni un'interazione con l'assistente vocale di Apple ("*Siri*")
- **GamePlayKit:** Framework che offre una serie di funzionalità e strumenti per semplificare l'implementazione di logiche di gioco complesse.

Come detto questi risultano essere una minima parte di quello che mette a disposizione Apple a livello di Framework. Di seguito sono riportati i framework che sono stati utilizzati per lo sviluppo dell'applicazione.

1.3.1 Swift

Swift [6] è un linguaggio di programmazione Object-Oriented utilizzato unicamente per lo sviluppo di applicazioni per sistemi operativi "made by Apple". Esso risulta essere un linguaggio molto potente, facile da imparare e viene costantemente aggiornato con numerose componenti (*frameworks*), grazie ai quali è possibile gestire le funzionalità di dispositivi che vengono col tempo rilasciati (*VisionKit*, per esempio, è un *framework* aggiuntivo da poco rilasciato per permettere ai *Developers* di sviluppare applicativi per il nuovo Apple Vision Pro). Essendo orientato agli oggetti, permette agli utenti di dividere il progetto in classi, oltre a garantire una totale compatibilità con i concetti di ereditarietà e polimorfismo. Una delle caratteristiche distintive di Swift è la sua sicurezza del tipo statico. Il compilatore di Swift esegue un'analisi completa del tipo durante la compilazione, rilevando potenziali errori di tipo e segnalando gli errori agli sviluppatori prima dell'esecuzione del programma. Ciò contribuisce a ridurre gli errori e a migliorare la robustezza e l'affidabilità delle applicazioni. Ovviamente l'intera applicazione si basa su questo linguaggio.

1.3.2 SwiftUI

SwiftUI [7] è un framework molto potente creato da Apple, che consente di gestire interfacce utenti per qualsiasi sistema operativo dell'Azienda; esso, come suggerisce il nome, è basato su Swift e consente di definire le metodologie con cui l'interfaccia deve apparire. Risulta infatti essere una valida alternativa a UIKit ed altri frameworks creati per lo stesso scopo. Per l'applicazione SwiftUI è stato il principale framework utilizzato per lo sviluppo delle interfacce e di notevole importanza sono state le variabili di Binding e di ambiente che mette a disposizione. Le variabili di ambiente in SwiftUI sono definite tramite il modificatore `.environment()` e sono ereditate in modo ricorsivo da tutte le viste figlie; Tra queste osserviamo:

EnvironmentObject: consente di condividere dati tra diverse viste senza passarli manualmente da una vista all'altra. Si crea un oggetto di ambiente utilizzando la proprietà `@EnvironmentObject` e lo si inserisce nella gerarchia delle viste utilizzando il modificatore `.environmentObject()`.

1.3.3 UIKit

UIKit [8] è un altro framework utilizzato per la realizzazione di interfacce, reso disponibile a partire dal 2007. Una tra le componenti più importanti messe a disposizione sono sicuramente i *viewController*, con i quali è possibile gestire degli eventi di vita, della presentazione di altre viste, dell'aggiornamento dell'interfaccia utente e della gestione delle interazioni dell'utente. Ogni schermata nell'applicazione può essere rappresentata da un **UIViewController**.

Altro componente di notevole importanza è la **UIView**, che è la classe base per tutti gli elementi dell'interfaccia utente in UIKit. Le UIView rappresentano i blocchi di base per la creazione dell'interfaccia utente, come pulsanti, etichette, campi di testo, immagini, viste di navigazione e molto altro. il componente UIKit nell'applicazione è stato usato solo per pochi casi in quanto essa è pienamente basata su SwiftUI.

1.3.4 GameplayKit

GameplayKit [5] è un framework che consente la gestione degli agenti e l'intelligenza artificiale, la gestione dei comportamenti dei personaggi e la simulazione di eventi casuali. Nel caso dell'applicazione mi è stato utile per la realizzazione del minigame "Gioca con Remy". In modo particolare grazie a questo framework è possibile implementare caratteristiche di gioco, movimenti, transizioni e interazioni. Il gioco che ho sviluppato tramite questo framework di fatto sfrutta a pieno tutte le funzionalità che mette a disposizione ed in particolare mi ha consentito di "dare vita" al personaggio e permettergli movimenti in base all'interazione con l'utente. I vari casi di movimento del personaggio che variano in base all'evento sono stati creati a livello grafico grazie a Photoshop che verrà descritto nel *Paragrafo 1.6*

1.4 XCode

Xcode [9] è l'ambiente di sviluppo integrato (IDE) ufficiale di Apple per la creazione di applicazioni iOS, macOS, watchOS e tvOS. L'IDE è interamente sviluppato da Apple, risulta essere reperibile sull'AppStore e necessita di alcuni componenti aggiuntivi per lo sviluppo prima di essere pienamente utilizzabile. Con il rilascio della versione 6 è stato aggiunto il nuovo linguaggio di programmazione **Swift**, presentato durante la WWDC14, inoltre introduce caratteristiche importanti come il Live Rendering, che permette di visualizzare gli oggetti in tempo reale mentre vengono sviluppati, così come verrebbero visualizzati in runtime. Le modifiche fatte alle viste possono essere ovviamente visualizzate mediante diverse tipologie di dispositivi ed anche con un vero e proprio simulatore con proprietà definite da Interface Builder e anche con dati fittizi per pre-popolare le UI in modo da poter avere esempi reali di come apparirà l'interfaccia grafica.

Le principali caratteristiche di Xcode includono:

- **Editor di codice:** Xcode fornisce un editor di codice avanzato con evidenziazione della sintassi, completamento automatico, correzione automatica e suggerimenti intelligenti per aiutare gli sviluppatori a scrivere codice in modo efficiente.
- **Debugger:** Xcode offre un potente debugger che consente agli sviluppatori di individuare e risolvere gli errori nel codice durante l'esecuzione dell'applicazione.
- **Strumenti di analisi delle prestazioni:** Xcode include strumenti per analizzare le prestazioni dell'applicazione, individuare eventuali problemi di velocità o utilizzo delle risorse e ottimizzare il codice per migliorare le prestazioni dell'app.
- **Simulatore:** Xcode include un simulatore iOS che consente agli sviluppatori di testare e debuggare le proprie applicazioni su diverse versioni di dispositivi iOS senza dover possedere fisicamente ogni dispositivo.

Xcode è uno strumento essenziale per gli sviluppatori di app iOS, offrendo una vasta gamma di funzionalità integrate che semplificano il processo di sviluppo e migliorano l'efficienza.

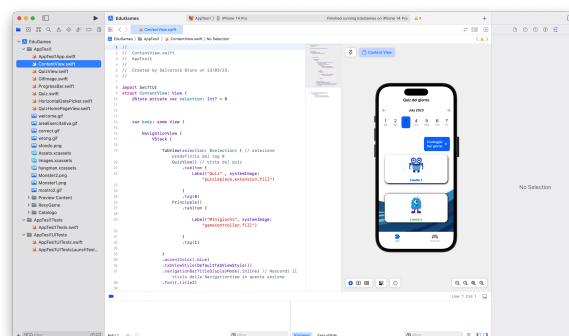


Figura 1.4: interfaccia di XCode

1.5 Sketch UI design

Sketch [15] è un software di progettazione grafica ampiamente utilizzato per la creazione di interfacce utente (UI) e il design di app. Garantisce un'interfaccia intuitiva che consente di utilizzare tutti i layout standard definiti da Apple da implementare per un'applicazione su sistema iOS. In particolare garantisce la presenza di ogni tipo di finestra dei prodotti Apple oltre alle icone ed i colori standard del sistema stesso.

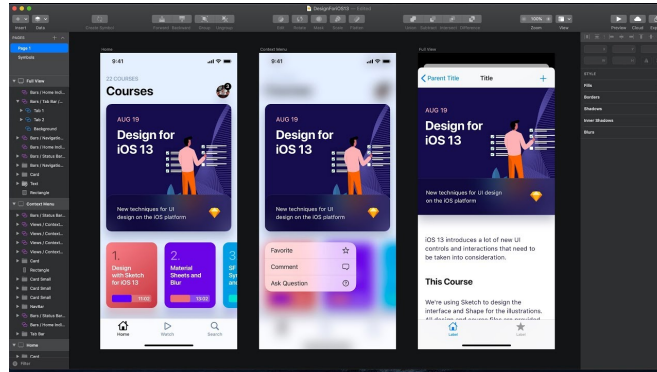


Figura 1.5: interfaccia generica di Sketch

Alcune delle caratteristiche principali di Sketch includono:

- **Creazione di layout:** Sketch offre strumenti per creare rapidamente layout che sono praticamente quelli standard di Apple e che non devono essere alterati date le politiche aziendali/
- **Librerie di risorse:** Sketch permette di creare librerie di risorse contenenti stili, icone e altri elementi di design comuni, che possono essere condivisi e utilizzati in diversi progetti.
- **Prototipazione interattiva:** Sketch consente di creare prototipi interattivi delle app, permettendo di visualizzare e testare le interazioni utente prima di iniziare lo sviluppo effettivo.

In generale l'applicazione risulta ben nota tra i designer oltre che essere di suo ben strutturata ed intuitiva, per cui il suo utilizzo è stato fondamentale per una prima bozza di quella che doveva essere l'interfaccia dell'applicazione

1.6 Photoshop

Photoshop [14] è un software di elaborazione di immagini che ha rivoluzionato il campo del design grafico grazie alla sua potenza e alle sue numerose funzionalità. È ampiamente utilizzato da professionisti creativi in vari settori, consentendo loro di manipolare,

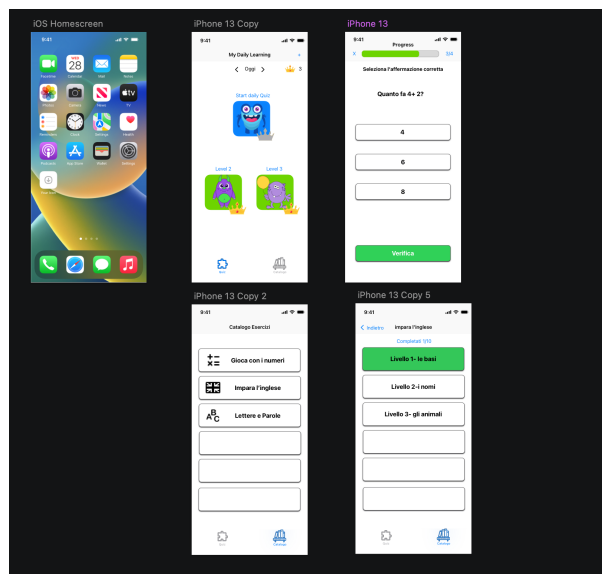


Figura 1.6: Sviluppo iniziale dell' interfaccia di Edugames

ridisegnare e migliorare le immagini in modi incredibili. Il software mette a disposizione una vasta gamma di strumenti, come maschere, livelli e selezioni, che permettono agli utenti di eseguire operazioni precise e dettagliate sulle immagini.

Per EduGames, Photoshop è stato uno strumento fondamentale nel processo di creazione del minigame "Gioca con REXY". Grazie alle sue potenti funzionalità, è stato possibile animare il personaggio principale, consentendo di creare movimenti realistici e di esprimere una varietà di emozioni in base agli eventi che si verificano nel gioco. Inoltre, il software è stato utilizzato per creare gli ostacoli, il cielo e il terreno di gioco, consentendo di dare vita a un ambiente coinvolgente e stimolante per i giocatori.

Nel caso dell'icona di EduGames, Photoshop è stato lo strumento principale utilizzato per la sua progettazione. Attraverso il software, sono state apportate varie ottimizzazioni e regolazioni alle immagini presenti all'interno dell'icona. Operazioni come il ritaglio, il ridimensionamento e l'aggiustamento dei colori sono state eseguite per garantire che le immagini fossero adatte alle dimensioni e alle specifiche richieste. Inoltre, l'utilizzo dei livelli ha permesso di creare un collage di immagini che riuscisse a riassumere in modo efficace il contenuto dell'applicativo, catturandone l'essenza in modo visivamente accattivante.

Durante il processo di creazione dell'icona, una fase importante è stata documentata attraverso uno screenshot. Questo mostra la fase finale della progettazione e della creazione dell'icona di gioco successivamente importata sul XCode.



Figura 1.7: Sviluppo dell'Icona

2.1 Modelli di progettazione software

La progettazione e lo sviluppo di un'applicazione non sono attività facili da portare avanti. Le attività fondamentali e comuni a tutti i processi software sono le seguenti (*Sommerville, Ing.Del Software*) :

- **specifica:** definizione di cosa deve fare il software e dei vincoli del suo sviluppo
- **sviluppo:** progettazione e programmazione
- **validazione:** verifica che il software prodotto sia conforme alle richieste del suo committente
- **evoluzione:** modifica del prodotto per adeguarlo ai requisiti dell'utente e del mercato che cambiano

In particolare, un modello di processo software è una descrizione semplificata di un processo (o di un suo aspetto) - osservato da un determinato punto di vista:

- **modello a flusso di lavoro (workflow):** il processo è una sequenza di azioni con relativi input, output e dipendenze
- **flusso di dati o modello di attività:** il processo è un insieme di attività, che modificano delle informazioni, come possono essere delle specifiche di progetto
- **modello ruolo/azione:** rappresenta i ruoli delle persone coinvolte e le attività di cui sono responsabili

Dunque un modello è una descrizione che permette di studiare quali problemi possono capitare durante la costruzione di un sistema e permette alle parti che lo compongono di comunicare tra di loro con una terminologia comune. Organizzazioni come Apple hanno un proprio processo di sviluppo.

Nel caso studio di mio interesse, il modello di sviluppo per un'Applicazione iOS è

stato uno dei primi argomenti affrontati del corso. Non risulta infatti scontato svolgere determinate operazioni preliminari prima di poter passare alla vera e propria fase di coding.

L'insieme dei passi che seguono compongono un modello che, se seguito correttamente permette di sviluppare passo dopo passo numerose idee e spunti per l'applicazione finale.

2.2 CBL

La Challenge Based Learning [3] , (*anche nota con l'acronimo CBL*) è un approccio educativo che si concentra sul coinvolgere gli studenti in problemi del mondo reale o "sfide" per sviluppare le loro abilità e conoscenze. Questo approccio mira a fornire agli studenti un contesto significativo per apprendere e incoraggia la collaborazione, la creatività e l'interazione con il mondo esterno.



Figura 2.1: CBL circle

Le fasi che compongono la CBL in particolare sono **tre** ed in particolare si distinguono in:

- **Engage**: fase iniziale che permette agli studenti di passare da una "Big Idea" ad una challenge concreta. Gli studenti sono incoraggiati a fare domande e condividere le loro conoscenze ,ciò può essere fatto attraverso l'esplorazione di storie, esperienze personali, video, dibattiti o presentazioni che pongono l'attenzione sul problema da affrontare. L'obiettivo è attivare la curiosità e stimolare il desiderio di comprendere meglio la questione. Durante il percorso di apple foundation e di tirocinio abbiamo infatti sviluppato video creativi che permettessero un primo approccio allo sviluppo del progetto.

- **Investigate:** fase intermedia in cui gli studenti pianificano e partecipano ad un viaggio che costruisce le fondamenta di una soluzione e indirizza a dei requisiti. Utilizzando domande guida, gli studenti conducono ricerche, interviste, analisi dei dati e altre attività investigative per acquisire conoscenze e affinare la loro comprensione del problema. Questa fase promuove il pensiero critico, l'analisi delle fonti e l'interpretazione dei dati. Per la fase iniziale di sviluppo dell'App abbiamo creato dei form che potessero fornirci informazioni sulle esigenze dei possibili utenti finali.
- **Act:** fase finale, basata sugli "indizi" , in cui gli studenti attuano quello che hanno appreso. Questa fase coinvolge la progettazione, la creazione di prototipi, l'implementazione di strategie o azioni concrete per affrontare il problema. Gli studenti possono collaborare in team, testare e migliorare le loro soluzioni, tenendo conto dei feedback ricevuti. L'obiettivo è applicare ciò che è stato appreso per sviluppare una soluzione significativa e tangibile.

2.3 Big Idea

La Big Idea è il concetto su cui si basa l'intero progetto e si può considerare come facente parte della fase di **Engage**. Nel particolare è un punto di partenza per l'apprendimento degli studenti che aiuta a comprendere il contesto e l'obiettivo della challenge stessa. Una Big idea può essere una domanda, una frase o un concetto che ci porta a stabilire un contesto significativo e stimolante per l'apprendimento, connettendo il tema della challenge con esperienze del mondo reale e incoraggiando gli studenti a esplorare, riflettere e sviluppare soluzioni creative. Nel caso di **EduGames** la big idea risiede nel concetto di **Apprendimento interattivo**, ovvero una forma di apprendimento che permetta di imparare tramite degli stimoli ed in particolare l'uso delle tecnologie digitali e delle piattaforme online ha notevolmente ampliato le possibilità di apprendimento interattivo, offrendo strumenti e risorse interattive che permettono agli studenti di esplorare, collaborare e interagire con il contenuto di apprendimento in modi innovativi e coinvolgenti. In base a questa Big Idea dunque nasce la proposta di qualcosa che possa racchiudere un insieme di componenti formativi e renderli interattivi.

2.4 Essential Question

L'Essential Question è il collegamento tra la Big Idea e le nostre vite; stimola la riflessione e l'approfondimento del pensiero critico. Si tratta di una domanda che va al cuore di un argomento o di una sfida e che invita gli studenti a indagare, analizzare e comprendere in modo più approfondito i concetti e le conoscenze legate a tale argomento. Nel nostro caso l'essential question è: **Come si può consentire a dei bambini di imparare (anche al di fuori dell'ambiente scolastico) divertendosi?** La domanda ovviamente non è di facile risposta, ma ci ha consentito di far fronte ad un problema non di banale importanza. Consentire ai bambini di imparare divertendosi li stimola ad esplorare, scoprire e acquisire conoscenze in modo autonomo e attraverso

esperienze ludiche e creative, possono apprendere in modo naturale e spontaneo, senza sentirlo come un impegno o un obbligo. Risulta importante sottolineare questo tipo di apprendimento non significa evitare sfide o sacrificare la qualità dell'apprendimento. Al contrario, l'apprendimento divertente dovrebbe essere progettato in modo da fornire stimoli intellettuali, incoraggiare il pensiero critico e la risoluzione di problemi. L'obiettivo è creare un ambiente in cui i bambini si sentano motivati, curiosi e desiderosi di esplorare nuove conoscenze e abilità.

2.5 The Challenge

La Challenge risulta essere una sfida o un problema che gli studenti devono risolvere. Nel particolare porta l'essential question alla "Call to action". La challenge permette agli studenti di stimolare la curiosità di apprendere mettendosi in gioco distaccandoli dal solo imparare mediante libri e portandoli a fare ricerche, a collaborare con i loro compagni di classe, a interagire con esperti del settore o membri della comunità, a raccogliere dati e a sperimentare diverse strategie. Durante il percorso di una challenge, gli studenti acquisiscono non solo conoscenze specifiche legate al tema trattato, ma anche competenze trasversali come la collaborazione, la comunicazione, il problem solving e la pensiero critico. Nel nostro caso la Challenge consiste nel **Progettare un "posto" in cui i bambini possano imparare divertendosi, al di fuori del contesto scolastico**. La sfida ovviamente sta nel progettare il tutto facendo in modo che non si esca troppo dal contesto, e si sfiori nel solo divertimento. Per far ciò, successivamente ad un attenta analisi e discussione (*oltre che ad una revisione di alcune risposte di utenti alle nostre domande*) sul da farsi, si è deciso di impostare il tutto realizzando un vero e proprio ambiente virtuale in cui i bambini possano espandere le loro conoscenze (anche mediante delle sfide), e allo stesso tempo possano divertirsi e migliorare le proprie capacità motorie di interazione.

Big Idea ➡ Essential Question ➡ Challenge

Figura 2.2: Passi della prima fase della CBL

2.6 Guiding Questions

Le guiding questions fanno parte della fase intermedia della CBL, ed in particolare consentono agli studenti di supportare il processo di apprendimento ponendosi domande, grazie alle quali è possibile stimolare il pensiero. Esse sono collegate all' "Essential Question" e alla "Big Idea". Le guiding questions possono essere aperte e generative, incoraggiando gli studenti a formulare risposte elaborate e a sviluppare il proprio pensiero. Possono anche essere più specifiche e mirate, focalizzandosi su concetti o dettagli particolari. Nel contesto che stiamo analizzando (CBL based), esse consentono a chi

ne fa uso di portare a compimento una challenge, o comunque di portarla in uno stadio avanzato. Le guiding questions aiutano anche a promuovere la riflessione e la metacognizione, incoraggiando gli studenti a considerare il loro processo di apprendimento, le strategie utilizzate e i risultati ottenuti.

In particolare le Guiding questions per la Big Idea precedentemente descritta nel nostro caso sono:

- Quali sono le principali sfide che i bambini affrontano nell'accesso a opportunità di apprendimento interattivo e creativo al di fuori delle aule scolastiche?
- Quali sono gli elementi chiave di un ambiente educativo interattivo e creativo?
- Come possiamo garantire l'inclusione e l'accessibilità degli spazi di apprendimento interattivi per i bambini con diverse abilità e background culturali?
- Come possiamo creare un ambiente motivante e stimolante che favorisca l'interesse e la curiosità dei bambini per l'apprendimento al di fuori delle aule scolastiche?
- Quali strategie pedagogiche possono essere adottate per facilitare l'apprendimento interattivo e creativo all'interno di questi spazi?
- Come possiamo integrare la tecnologia in modo efficace per arricchire l'apprendimento interattivo e creativo al di fuori delle aule scolastiche?

In definitiva, le guiding questions svolgono un ruolo fondamentale nel processo di apprendimento basato sulla CBL, consentendo agli studenti di sviluppare il pensiero critico, rispondere all'Essential Question e avanzare nella challenge. Nel nostro caso ci hanno permesso di capire come avanzare con il progetto e su quale approccio adottare per fornire un corretto metodo di apprendimento.

2.7 Guiding Activities

Le "Guiding Activities" sono delle attività svolte al di fuori del contesto di studio che sono strettamente correlate alla "Big Idea", all'"Essential Question" e alle "Guiding Question". Queste attività sono strutturate in modo da offrire agli studenti un percorso guidato e progressivo, fornendo indicazioni specifiche su cosa fare, come fare e quali risorse utilizzare durante il processo di apprendimento. Durante il percorso, sono i docenti stessi a fornire questo tipo di attività.

Personalmente, è stato utile presentare la challenge mediante un video che racchiudesse le problematiche coinvolte nell'idea sviluppata. Tuttavia, in generale, le "Guiding Activities" possono assumere varie forme a seconda del contesto e possono spaziare da video a comuni attività esplorative che permettono agli studenti di progredire nel progetto. Possono essere modificate o personalizzate in base al livello di competenza degli studenti, alle loro conoscenze pregresse e agli interessi individuali.

Guiding Question → Guiding Activities

Figura 2.3: Passi della seconda fase della CBL

Nel nostro progetto, le guiding activities che hanno favorito il lavoro sono state le seguenti:

- **Ricerca sulle esigenze dei bambini:** Abbiamo eseguito ricerche e sondaggi per capire quali sono le materie di cui i bambini hanno bisogno. Questo è stato fatto prevalentemente tramite i genitori e persone che sono del settore. La raccolta di tali informazioni è risultata molto utile, in quanto ci ha permesso di ricavare delle idee circa i quesiti da proporre.
- **Confronto con studenti di facoltà specializzate:** Abbiamo fatto dei confronti con colleghi della facoltà di Scienze della formazione primaria, da cui sono scaturite varie esigenze, tra cui lo sviluppo di competenza matematica e competenza di base in scienza e tecnologie. Si sostiene che la tecnologia in aula aiuta a motivare migliorando la predisposizione all'apprendimento, il processo educativo fondamentale per il percorso di crescita [10].
- **Confronto sul campo:** Abbiamo fatto delle osservazioni sul campo, grazie alle quali abbiamo potuto avere una visione concreta della realtà di interesse. Abbiamo valutato personalmente la reazione dei bambini a determinati quesiti, e ciò ci ha permesso di capire quali fossero gli argomenti più e meno amati.

Queste attività hanno contribuito a fornire una base solida per la progettazione del percorso di apprendimento, consentendoci di comprendere meglio le esigenze dei bambini e di adattare le attività di insegnamento di conseguenza.

Le "Guiding Activities" sono delle attività svolte al di fuori del contesto di studio. Esse sono sempre strettamente correlate alla "Big Idea" all' "Essential Question" e alle "Guiding Question", esse sono strutturate in modo da offrire agli studenti un percorso guidato e progressivo, fornendo indicazioni specifiche su cosa fare, come fare e quali risorse utilizzare durante il processo di apprendimento. Durante il percorso sono i docenti stessi a fornire questo tipo di attività. Personalmente ci è stato utile come attività presentare la challenge mediante un video che racchiudesse le problematiche coinvolte nell'idea sviluppata.

2.8 Guiding Resources

Le guiding resources, sono strumenti utilizzate per supportare gli studenti nel processo di apprendimento e nell'affrontare una challenge. Queste risorse sono selezionate e fornite agli studenti per guidarli, offrendo informazioni, strumenti, esempi o modelli che possono essere utili nel contesto della challenge. Esse sono sempre strettamente legate al concetto di "Big Idea", "Essential Question" e "Guiding Question" ; sono ovviamente

dei materiali mirati che forniscono a chi ne fa uso una vasta gamma di informazioni e strumenti utili per esplorare, approfondire e collegare concetti nell'ambito della Big Idea e delle guiding questions.

Nel caso in questione le guiding resources utili a tale scopo sono state:

- **Materiale didattico fornito da colleghi di facoltà specializzate:** Documenti ufficiali forniti da colleghi universitari
- **Strumenti digitali e software:** Strumenti che hanno aiutato la comprensione del metodo di apprendimento interattivo
- **Esperti del settore:** Persone che praticano il mestiere da anni e che garantiscono una guida efficiente nel settore

2.9 Synthesis

La big idea in questo contesto riguarda la mancanza di opportunità di apprendimento coinvolgente e creativo per i bambini al di fuori delle aule scolastiche. Questo solleva il quesito che ci porta a chiederci come realizzare spazi educativi al di fuori del contesto scolastico che consentano ai bambini di apprendere divertendosi. La sfida principale risiede proprio nello sviluppare un contenuto divertente e coinvolgente, ma allo stesso tempo che possa essere utile ai fini educativi. Da ciò che è stato ricavato è emerso che questo metodo di apprendimento risulterebbe molto efficiente, soprattutto nei casi in cui il bambino non risulta propenso allo studio vero e proprio. I bambini poi di per se sono amanti delle sfide e il più delle volte si lasciano trasportare senza pensare che in effetti imparano e giocano allo stesso tempo. Ovviamente svolgere questo tipo di sfide in un aula fisica sarebbe molto complicato da realizzare e non sarebbe poco il materiale da mettere a disposizione (*Da un confronto con studenti di facoltà specializzata*). Per creare un ambiente che possa essere motivante ed allo stesso tempo utile all'apprendimento sono necessarie dunque nuove tecnologie "creative" che sviluppino la curiosità nel bambino. Questo può avvenire solo con l'apporto della tecnologia a tale ambito, in modo da arricchire l'apprendimento esterno all'ambiente comune ed ormai standard delle scuole. Ovviamente risulta banale dire che i quesiti che vanno messi a disposizione debbano essere qualitativamente appropriati e con un giusto grado di difficoltà.

2.10 Soluzione

La soluzione al problema affrontato rappresenta il metodo con cui si è deciso di risolvere la questione. Nel caso in oggetto, una soluzione per colmare questa mancanza è quella di creare un'applicazione **di apprendimento interattivo per bambini**, che permetta loro di continuare ad apprendere al di fuori dell'ambiente scolastico senza risultare eccessivamente gravosa. L'applicazione deve essere appositamente studiata per i bambini, quindi l'interfaccia (che è stata generata inizialmente utilizzando appositi strumenti grafici e poi con l'IDE effettivo) deve risultare semplice ed intuitiva. Devono essere introdotti dei quiz, opportunamente suddivisi per livello di difficoltà e con sistema di

punteggio. Inoltre, è possibile includere giochi educativi relativi a materie comuni come matematica e inglese, e dei minigiochi che consentano di sviluppare un'interazione più coinvolgente. Le domande stimolanti e interattive sfideranno i bambini a rispondere correttamente, accumulando punti per aumentare la motivazione e l'impegno nell'apprendimento. I giochi menzionati possono essere utilizzati per l'apprendimento delle diverse materie e anche per stimolare la creatività del bambino. I minigiochi, invece, possono rappresentare ottimi ambienti virtuali che permette ai bambini di reagire a determinati eventi e di ragionare per affrontare le sfide proposte. In campo educativo questa soluzione rappresenta un buon contributo.

Research synthesis ➔ **Solution concept**

Figura 2.4: Fase finale approccio CBL

l'insieme delle ricerche attuate e delle domande che sono state poste ai fini progettuali, fino ad arrivare alla soluzione definitiva vengono infine riportate in un documento (*Nota come Canvas*) che mantiene tutti i dettagli della progettazione.

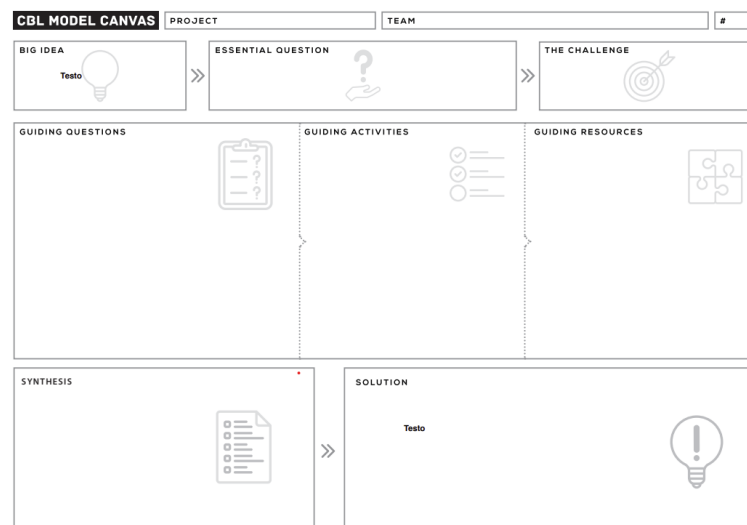


Figura 2.5: Modello di Canvas CBL

CAPITOLO 3

IMPLEMENTAZIONE E DESCRIZIONE DELLE FUNZIONALITÀ

3.1 Fasi di implementazione e Metodologie di approccio

La fase di implementazione segue immediatamente quella di progetto, in particolare, ci troviamo davanti al punto di svolta nel percorso dello stesso, tale che i concetti teorici e tutte le possibili soluzioni ricavate debbano essere messe in pratica. L'obiettivo è dunque quello di sviluppare un'applicazione che possa, nel suo piccolo, essere di supporto apprendimento infantile e possa incentivare col tempo i bambini ad appassionarsi allo studio e accrescere il loro livello di curiosità. Ovviamente la fase di implementazione prevede lo sviluppo sia della parte **front-end** che della parte di **back-end**, in particolare [4] :

- **Front-end**: si riferisce alla parte visibile e interattiva dell'applicazione che viene visualizzata dagli utenti. È ciò che essi vedono e con cui interagiscono direttamente, come per esempio l'interfaccia.
- **Back-end**: si riferisce alla parte dell'applicazione che gestisce ciò che essa svolge in background e si occupa delle funzionalità che supportano il front-end. Questa parte è responsabile della gestione dei dati, della logica dell'applicazione, delle operazioni del server e dell'interazione servizi esterni.
- **Full-Stack**: si riferisce al fatto di avere una copertura completa da parte di una singola persona sull'intero percorso di sviluppo di funzionalità (sia front-end che back-end). Nel caso in questione, ovviamente, l'approccio da me adottato è stato quest'ultimo.

Entrambe le fasi sono state ovviamente gestite durante il percorso di implementazione. In particolare, la parte di progettazione grafica (*fase che si colloca principalmente nel front-end dello sviluppo*), è stata completamente gestita da me oltre che alle fasi di programmazione che hanno riguardato alcune delle funzionalità dell'applicazione.

Al fine di garantire il successo dell'implementazione verranno seguite buone pratiche di sviluppo software, questo permette di garantire un controllo sullo sviluppo dell'applicazione oltre che di facilitare il lavoro di collaborazione.

I requisiti delineati nella fase di progettazione saranno interamente tradotti in linee di codice, in modo da generare un'interfaccia coinvolgente.

Il linguaggio di programmazione che è stato utilizzato (*Ampiamente descritto nel Capitolo 2*) è **Swift**, linguaggio proprietario Apple, e l'**IDE** di sviluppo è stato **XCode** (*Sempre sviluppato da Apple*).

3.2 Interfaccia

La progettazione dell'interfaccia utente è stata un punto cruciale nell'implementazione dell'applicazione. Senza una base di design solida, diventa complicato creare l'interfaccia scrivendo direttamente codice. Pertanto, ho ritenuto opportuno iniziare disegnando l'applicazione utilizzando **Sketch** (un tool grafico descritto nel Paragrafo 1.5), seguendo naturalmente tutte le linee guida necessarie per un'applicazione iOS, tra cui:

- **Integrità estetica:** ossia quanto l'aspetto e il comportamento di un'applicazione si integrino con la sua funzione.
- **Consistenza:** un'applicazione coerente deve garantire che elementi con funzioni simili appaiano in modo simile.
- **Layout:** un'applicazione iOS deve rispettare un layout che consenta alle persone di utilizzarla su tutti i loro dispositivi in qualsiasi contesto.

Il mio lavoro si è quindi basato sul rispetto di questi principi. Dopo aver definito i concetti di integrità estetica, consistenza e layout, ho utilizzato **Sketch** per creare i mockup dell'interfaccia utente dell'applicazione iOS. Questo strumento grafico mi ha permesso di progettare e visualizzare in modo intuitivo le schermate, compresi i layout, gli elementi grafici, le icone e i colori.

Durante la progettazione dell'interfaccia utente, ho prestato particolare attenzione all'usabilità dell'applicazione. Ho cercato di rendere l'interfaccia intuitiva e facile da utilizzare per gli utenti, tenendo conto delle convenzioni e delle linee guida di design di iOS. Ciò significa che ho adottato elementi familiari agli utenti di iOS, come le barre di navigazione, le schede e le icone riconoscibili.

Mi sono assicurato che venisse mantenuta coerenza visiva, in modo che gli elementi con funzioni simili fossero rappresentati in modo uniforme in tutta l'interfaccia. Questo crea un'esperienza utente più fluida e familiare, in cui gli utenti possono facilmente comprendere e navigare nell'applicazione senza dover imparare nuovi schemi di design per ogni schermata.

3.3 Divisione in Classi

Al fine di discutere l'implementazione delle funzioni da me gestite, è importante specificare che l'intero codice del progetto è stato opportunamente suddiviso in classi, ciascuna delle quali fornisce una funzione specifica all'interfaccia principale quando richiamata correttamente. Questo lavoro, sebbene possa sembrare banale, si è rivelato estremamente utile in termini di pulizia del codice e facilità di interpretazione di alcune funzioni, oltre ad essere una pratica buona e consigliata.

I vantaggi apportati da tale approccio sono numerosi e risiedono nella gestione e nell'organizzazione del progetto. Prima di tutto, ha contribuito a mantenere il codice più pulito e leggibile, separando le diverse funzionalità in unità distinte. Questo rende più semplice la comprensione e la manutenzione del codice nel lungo termine. La suddivisione in classi ha favorito la riutilizzabilità del codice in quanto ognuna di esse è vista come componente autonomo, che può essere richiamato e utilizzato in diverse parti dell'interfaccia principale o in altre sezioni del progetto. Questo permette di evitare la duplicazione del codice e di promuovere una struttura modulare ed efficiente.

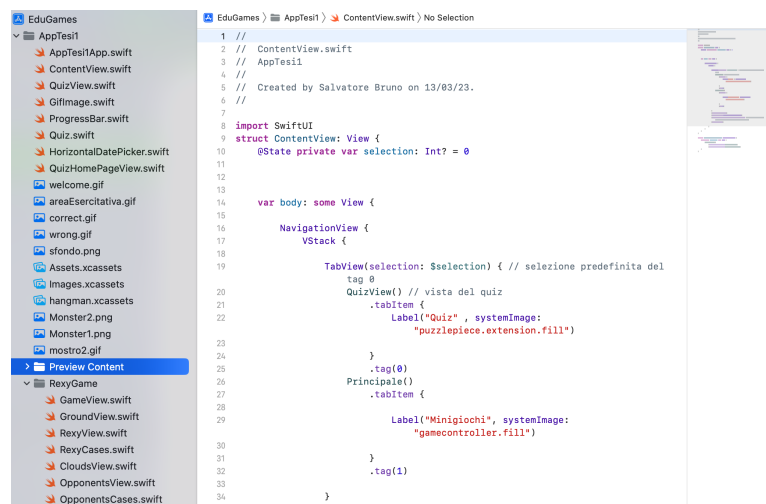


Figura 3.1: Divisione in classi in XCode

Per cui la ContentView (*Classe che riguarda l'interfaccia principale dell'applicazione*) non fa altro che richiamare le varie classi in modo da unire i componenti e generare l'interfaccia principale.

3.4 ContentView

La classe "ContentView" rappresenta la vista principale dell'interfaccia dell'applicazione. La sua implementazione si basa sul richiamo della classe "QuizView", che gestisce le varie funzioni legate al quiz. La "ContentView" include una "NavigationView", che offre la possibilità di navigare tra le diverse sezioni dell'applicazione e gestire la gerarchia delle viste. Seguendo gli standard Apple, sono stati aggiunti due "TabI-

tems” che consentono di passare dalla vista principale alla vista secondaria in modo intuitivo e rapido.



Figura 3.2: Schermata principale dell'app EduGames

La vista principale, come mostrato nella figura 16, include un calendario interattivo che si aggiorna automaticamente in base alla data corrente. Questo calendario consente agli utenti di sfogliare i giorni e i mesi precedenti per visualizzare i punteggi associati a specifiche date. L'utilizzo di un calendario interattivo offre un modo intuitivo per esplorare i risultati dei quiz in base alle diverse date.

Al centro della vista principale sono presenti i quiz di varie difficoltà, che gli utenti possono completare. Questi quiz rappresentano un elemento chiave dell'app EduGames, in quanto offrono l'opportunità di testare le proprie conoscenze e competenze in modo interattivo e coinvolgente. L'utilizzo di una classe dedicata come "QuizView" per

gestire le funzioni dei quiz consente una gestione organizzata e modulare delle attività di quiz all'interno dell'applicazione.

Completano la vista principale una sezione in cui viene visualizzato il punteggio totalizzato nell'arco della giornata corrente, fornendo agli utenti un resoconto immediato delle loro performance, e i due "TabItems" opportunamente creati con i loghi che li rappresentano. Questi elementi di navigazione facilitano l'accesso rapido e intuitivo alle diverse sezioni dell'applicazione, consentendo agli utenti di spostarsi tra le diverse funzionalità in modo fluido.

Complessivamente, la classe "ContentView" svolge un ruolo centrale nell'interfaccia dell'app EduGames, offrendo una vista principale che combina calendario interattivo, quiz, punteggi e navigazione fluida. Questa implementazione mira a fornire agli utenti un'esperienza piacevole e intuitiva durante l'utilizzo dell'applicazione, favorendo l'apprendimento interattivo e la partecipazione attiva.

3.5 HorizontalDatePicker

La classe "**HorizontalDatePicker**" è stata implementata per personalizzare il già presente DatePicker di SwiftUI. Il suo scopo principale è consentire la visualizzazione di un calendario giornaliero, in cui è possibile selezionare date antecedenti al fine di visualizzare i progressi passati.

Come mostrato nella Figura 17, il "HorizontalDatePicker" offre una vista orizzontale delle date, che consente agli utenti di scorrere attraverso i giorni precedenti e selezionare la data desiderata. Questo approccio consente un accesso rapido alle informazioni storiche e ai progressi passati, offrendo una panoramica completa dell'andamento nel tempo.

La classe "HorizontalDatePicker" è stata progettata per essere intuitiva e di facile utilizzo. Gli utenti possono semplicemente scorrere le date orizzontalmente per visualizzare i progressi in base alle date precedenti. La selezione di una data consente di ottenere informazioni specifiche su quel giorno e accedere ai relativi progressi o dettagli.



Figura 3.3: HorizontalDatePicker

L'obiettivo finale della classe "HorizontalDatePicker" è fornire agli utenti un modo semplice e intuitivo per navigare attraverso le date e accedere alle informazioni desiderate per monitorare i progressi nel tempo. Questa personalizzazione del DatePicker di Swift

tUI amplia le funzionalità di base offerte dalla libreria standard, offrendo un'esperienza utente migliorata e più flessibilità nella gestione delle date e dei progressi all'interno dell'applicazione EduGames.

3.6 QuizView

La classe "QuizView" rappresenta la vista principale dell'applicazione e svolge diverse funzioni chiave per offrire un'esperienza utente completa e coinvolgente. Vediamo nel dettaglio le principali funzioni gestite dalla classe:

- **HorizontalDatePicker:** La classe "QuizView" utilizza la classe "HorizontalDatePicker" per visualizzare un calendario nella parte superiore della vista. Questo componente consente agli utenti di selezionare una data specifica e visualizzare i quiz disponibili per quel giorno. L'inclusione del calendario nella vista offre una navigazione intuitiva attraverso le diverse date e rende facile per gli utenti trovare e accedere ai quiz giornalieri.
- **Punteggio:** All'interno della classe "QuizView" sono state implementate funzioni per il calcolo e la gestione del punteggio dei giocatori. Durante il completamento del quiz, il punteggio viene calcolato in base alle risposte corrette e incorrette dell'utente. La classe si occupa anche del salvataggio [2] del punteggio, utilizzando una funzione dedicata per archiviare i risultati ottenuti. Questo consente agli utenti di monitorare i propri progressi e di confrontare i punteggi con quelli degli altri giocatori.
- **Quiz:** La classe "QuizView" gestisce anche la visualizzazione dei quiz sulla schermata principale dell'applicazione. I due possibili quiz giornalieri, con diversi livelli di difficoltà, vengono mostrati agli utenti, offrendo loro la scelta di quale quiz completare. Utilizzando il componente di navigazione "NavigationLink", gli utenti possono selezionare un quiz e passare alla vista del quiz effettiva, richiamando la classe "QuizHomePageView". Questa vista offre un'interfaccia interattiva che permette di completare il quiz rispondendo alle domande proposte.

Durante tutto il processo di implementazione, ho dato grande importanza alle esigenze degli utenti finali, come insegnanti, genitori e bambini stessi. Ho coinvolto questi stakeholder chiave nelle fasi di progettazione e sviluppo, raccogliendo feedback preziosi per ottimizzare l'esperienza utente e garantire che l'applicazione soddisfi le loro aspettative.

3.7 ProgressBar

La barra di progresso presente nella vista relativa al Quiz è stata una personalizzazione che ho implementato per offrire agli utenti una rappresentazione visiva del progresso nel completamento del Quiz. La barra di progresso avanza in modo proporzionale al numero di domande presenti nel Quiz, consentendo agli utenti di avere un'idea chiara di quanto hanno completato e di quanto ancora devono affrontare.

L'implementazione di questa barra di progresso è stata realizzata utilizzando un doppio "Z-Stack", che è un contenitore in SwiftUI che consente di sovrapporre diversi elementi grafici. Nel nostro caso, abbiamo utilizzato due rettangoli: uno di sfondo e uno di riempimento.

Il rettangolo di sfondo rappresenta l'intera lunghezza della barra di progresso e rimane statico. Il rettangolo di riempimento, invece, è posizionato sopra il rettangolo di sfondo e rappresenta la parte della barra che viene progressivamente riempita mano a mano che il Quiz avanza.

La larghezza del rettangolo di riempimento viene calcolata in base alla percentuale di avanzamento del Quiz. Ad esempio, se il Quiz è composto da 10 domande e l'utente ha completato correttamente 5 domande, la larghezza del rettangolo di riempimento sarà il 50% della larghezza totale del rettangolo di sfondo.

Questa personalizzazione della barra di progresso offre un feedback visivo immediato sull'avanzamento del Quiz. Gli utenti possono vedere chiaramente quanto hanno completato e quanto ancora devono fare. Questo aiuta a mantenere alta la motivazione durante il Quiz e fornisce una chiara indicazione del tempo rimanente o del progresso compiuto.



Figura 3.4: Barra di progresso

3.8 GifImage

La classe "GifImage" è stata implementata all'interno del contesto di un'applicazione, specificamente come componente per la gestione delle immagini animate in formato GIF. Questa classe rappresenta un'importante componente nell'interfaccia utente e svolge un ruolo cruciale nel fornire un'esperienza visiva coinvolgente per gli utenti.

La classe è stata progettata seguendo le migliori pratiche di sviluppo per l'integrazione di GIF all'interno di un'applicazione iOS, utilizzando il framework SwiftUI. Essa implementa il protocollo `UIViewRepresentable`, che permette di incorporare una vista UIKit all'interno di SwiftUI, consentendo una perfetta integrazione con il sistema di sviluppo.

All'interno della classe GifImage, è presente una proprietà privata chiamata `name`, che rappresenta il nome del file GIF da visualizzare. Questa proprietà viene inizializzata attraverso il costruttore della classe, che accetta come parametro il nome del file GIF. Il metodo `makeUIView` è responsabile della creazione e configurazione di una vista `WKWebView`, una classe fornita dal framework WebKit, utilizzata per visualizzare contenuti web. All'interno di questo metodo, viene creata un'istanza di `WKWebView` e viene caricato il contenuto dell'immagine GIF utilizzando il nome del file fornito.

Viene effettuato un controllo per verificare se il file GIF esiste nel bundle dell'applicazione e, se presente, viene caricato nel WKWebView utilizzando il metodo `load`. L'URL di base per il caricamento dell'immagine viene ottenuto tramite il metodo `url.deletingLastPathComponent()`, che restituisce l'URL della cartella contenente il file GIF.

Il metodo `updateUIView` viene chiamato quando la vista viene aggiornata. In questa implementazione, viene semplicemente richiamato il metodo `reload()` del WKWebView, che forza il ricaricamento del contenuto.

La classe `GifImage` viene utilizzata all'interno dell'applicazione per gestire la visualizzazione di immagini GIF animate all'interno dell'interfaccia utente. Grazie alla sua integrazione con SwiftUI e all'utilizzo di WebKit, essa permette una gestione ottimale delle immagini animate, fornendo un'esperienza visiva coinvolgente per gli utenti dell'applicazione.

3.9 QuizHomePageView

La classe **"QuizHomePageView"**, gestisce a pieno la parte del quiz. In particolare la mia implementazione prevede la presenza del quesito e di un bottone che permette di avanzare alla domanda successiva.

Tutti i quesiti verranno estratti in maniera casuale dalla classe **"Quiz"** utilizzando una variabile *@StateObject*, che crea un oggetto di stato condiviso all'interno della vista. Questo oggetto può essere quindi utilizzato per mantenere e modificare lo stato dell'applicazione. La struttura di tale classe sarà meglio analizzata nel sottoparagrafo successivo (3.6.1).

Le immagini animate (*.gif*) presenti all'interno della schermata sono a loro volta gestite dalla classe **"GifImage"**; la barra di progresso invece è gestita dalla classe **"ProgressBar"**, che verrà analizzata nel *paragrafo 3.7*

All'interno della **"QuizHomePageView"**, ho progettato l'interfaccia per presentare in modo efficace il quesito al partecipante. Utilizzando un layout intuitivo, l'utente può visualizzare chiaramente il quesito e le opzioni di risposta. Ho prestato attenzione ai dettagli visivi e all'usabilità, al fine di garantire che la presentazione del quesito sia chiara e facilmente comprensibile per i partecipanti.

Nel contesto del quiz, ho stabilito che il numero totale di domande sia di 5. Ogni risposta corretta viene valutata con 20 punti. Questo sistema di punteggio è stato implementato per fornire una misura oggettiva delle prestazioni degli utenti durante il quiz. Inoltre, il punteggio aggiornato viene mostrato agli utenti, consentendo loro di tenere traccia dei progressi nel corso del quiz e creando una sfida intrinseca per ottenere un punteggio più alto.

Una volta giunti alla fine del quiz, sarà possibile tornare alla schermata principale, dove il punteggio conseguito sarà sommato al punteggio totale della giornata. Nella seconda view il punteggio totale comprenderà non solo il punteggio ottenuto nel quiz appena



Figura 3.5: Interfaccia del Quiz

completato, ma anche tutti i punteggi precedenti accumulati durante le altre sessioni di gioco. In altre parole, i sistemi si sincronizzano in modo da fornire un'immagine completa e accurata del punteggio totale.

3.9.1 Quiz

La classe "Quiz" è stata progettata in modo da essere facilmente osservabile attraverso il pattern di programmazione "Observable". Essa è una sottoclasse di "ObservableObject", il che significa che è possibile monitorare e ricevere notifiche sulle modifiche dei dati al suo interno.

All'interno della classe "Quiz", ho implementato una struttura chiamata "Question" che gestisce singolarmente ogni domanda del quiz. La struttura "Question" è caratterizzata da diversi attributi, tra cui il testo della domanda, un array di risposte possibili, l'indice della risposta corretta e un livello di difficoltà associato alla domanda.

```
init() {
  let allQuestions = [
    Question(text: "Qual è il colore del sole?", answers: ["Rosso", "Giallo", "Blu"], correctAnswer: 1, difficulty: 1),
    Question(text: "Quanti giorni ci sono in una settimana?", answers: ["5", "6", "7"], correctAnswer: 2, difficulty: 2),
    Question(text: "Quale animale fa il verso 'miao'?", answers: ["Cane", "Gatto", "Uccello"], correctAnswer: 1, difficulty: 1),
  ]
}
```

Figura 3.6: Codice del costruttore

Nel costruttore della classe "Quiz", ho inizializzato un array di domande disponibili. Questo array viene successivamente mescolato utilizzando il metodo "Shuffled", in modo che ogni volta che il quiz viene avviato, le domande vengano presentate in ordine casuale. Questo approccio garantisce una maggiore varietà e imprevedibilità nel quiz, rendendo l'esperienza di gioco più interessante.

Inoltre, ho utilizzato il metodo "prefix(5)" sull'array di domande per ridurre il sottoinsieme di domande da mostrare a cinque. Questo significa che, all'avvio del quiz, verranno selezionate casualmente cinque domande dalle domande disponibili. Questo aiuta a mantenere il quiz di dimensioni gestibili e ad assicurare che l'utente non venga sopraffatto da un numero eccessivo di domande.

La struttura del codice presentata nella figura mostra il costruttore della classe "Quiz" e le operazioni iniziali eseguite per preparare il quiz. Tuttavia, il codice può continuare ad essere sviluppato ulteriormente con altri metodi e funzionalità per gestire il flusso del quiz, calcolare i punteggi, gestire le risposte degli utenti e altro ancora.

3.10 RexyGame

Rexy Game non è una singola classe, bensì è un vero e proprio minigame. L'implementazione di quest'ultimo è partita dalla progettazione dell'interfaccia di gioco, del personaggio e degli ostacoli. Lo sviluppo di ciò è avvenuto mediante il software "Photoshop" (*citato nel paragrafo 1.5*)

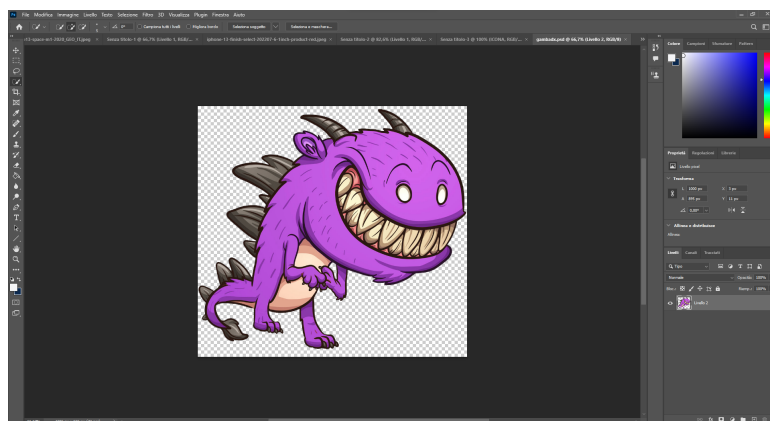
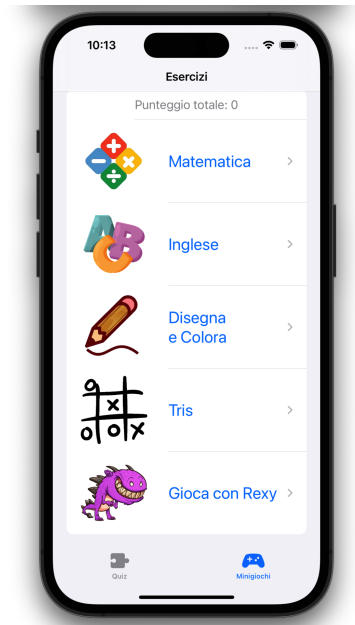


Figura 3.7: Frame gamba Dx

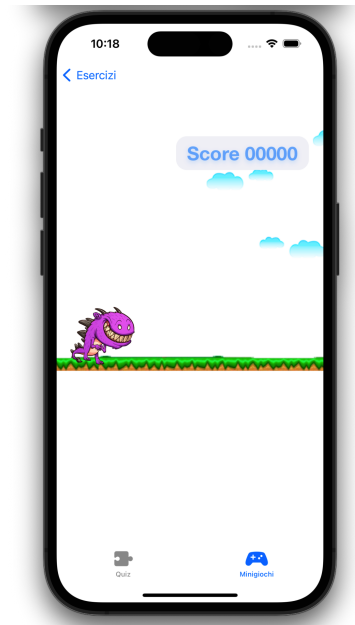
Dalla figura si denota che stavo generando il frame del passo della gamba destra del personaggio. Ovviamente è stato necessario generare tutto il movimento frame dopo frame, oltre che agli ostacoli e al terreno di gioco.

Successivamente, l'implementazione effettiva del minigioco REXY Game è avvenuta attraverso ovviamente *SwiftUI* con l'utilizzo del framework *GameplayKit*, con il quale è stato possibile sviluppare determinate fasi di gioco. Il codice è stato utilizzato per gestire la logica di gioco, come le interazioni tra il personaggio e gli ostacoli, la gestione degli

input utente ed il punteggio. Per quanto riguarda il punteggio, è stata implementata una variabile apposita che consente il salvataggio e la somma del punteggio. Questo avviene sia nella vista principale, dove viene considerato come punteggio giornaliero, sia nella vista secondaria, dove viene considerato come punteggio totale accumulato dall'installazione del gioco.

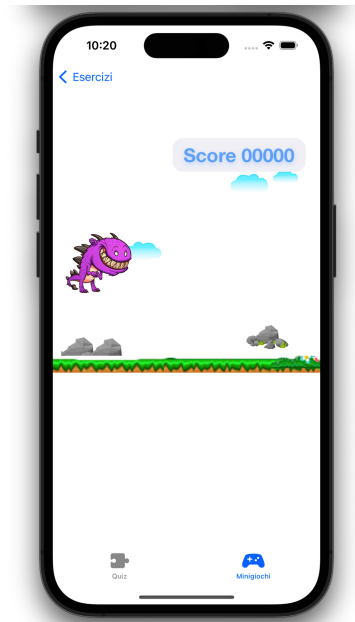


(a) Seconda view

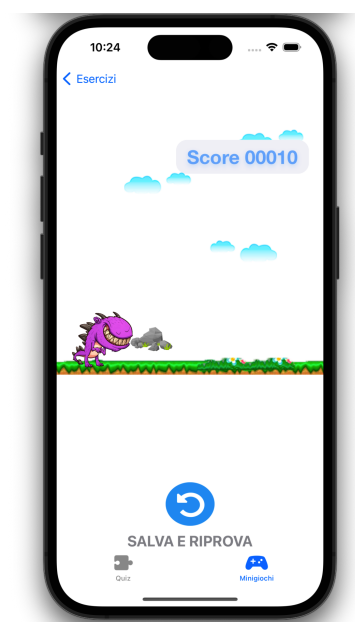


(b) Figura 23: Fase di camminata

Figura 3.8: Fasi Iniziali



(a) Fase di salto



(b) Game Over

Figura 3.9: Fasi di gioco

L'implementazione dell'intero gioco è avvenuta mediante 7 classi, ognuna delle quali gestisce una funzionalità dello stesso.

Una delle classi fondamentali coinvolte nell'implementazione del gioco REXY Game è responsabile della gestione delle "hitbox" degli ostacoli. La hitbox rappresenta un'area invisibile intorno agli ostacoli con cui il personaggio del gioco può interagire. Questa classe specifica la posizione, le dimensioni e le interazioni tra la hitbox degli ostacoli e il personaggio principale. Attraverso questa gestione, il gioco è in grado di rilevare quando il personaggio entra in contatto con un ostacolo e di intraprendere le azioni necessarie, come ad esempio terminare la partita in caso di collisione. Questo aspetto è fondamentale per garantire la corretta dinamica di gioco e creare una sfida interessante per i giocatori.

Oltre alla classe responsabile della gestione delle hitbox degli ostacoli, il gioco REXY Game coinvolge anche altre classi che svolgono ruoli chiave nel funzionamento del gioco. Queste classi sono state implementate per gestire il movimento del terreno, del personaggio, degli ostacoli e delle nuvole nel cielo. Ciascuna classe ha il compito di gestire specifiche funzionalità del gioco, consentendo il movimento fluido e realistico degli elementi all'interno dell'ambiente di gioco.

Ad esempio, la classe responsabile del movimento del terreno si occupa di generare e far scorrere il terreno sotto il personaggio in modo da creare l'effetto di avanzamento. La classe del personaggio gestisce il movimento del personaggio principale, inclusi il salto e il controllo della gravità. Le classi degli ostacoli si occupano di generare e posizionare gli ostacoli all'interno del gioco, nonché di controllare le collisioni con il personaggio. Infine, la classe delle nuvole nel cielo gestisce la generazione e il movimento delle nuvole nell'ambiente di gioco, aggiungendo un elemento visivo e atmosferico all'esperienza di gioco complessiva.

L'implementazione del gioco REXY Game coinvolge quindi un insieme di sette classi, ognuna dedicata a una specifica funzionalità del gioco. Queste classi lavorano sinergicamente per gestire il movimento del personaggio, le interazioni con gli ostacoli, il conteggio del punteggio e altre funzioni chiave del gioco. Questo approccio modulare e organizzato contribuisce a creare un'esperienza di gioco coinvolgente e divertente, incoraggiando i giocatori a migliorare i loro movimenti e le loro reazioni durante il gameplay.

Di seguito è riportata la sottocartella del progetto che contiene il gioco e parte del codice che si occupa di gestire la fase di salto del personaggio.

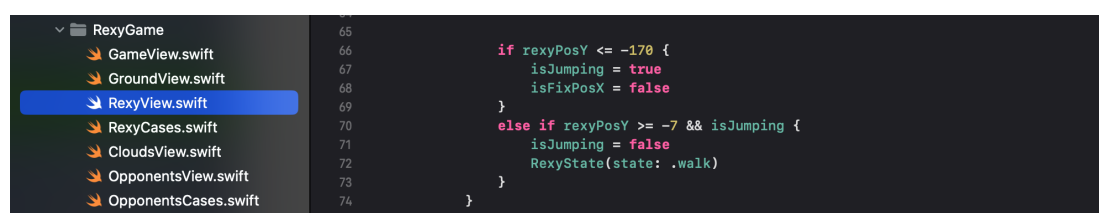


Figura 3.10: Schema delle classi del gioco e parte del codice

Questo studio ha cercato di rispondere al problema della **progettazione un "posto" in cui i bambini possano imparare divertendosi, al di fuori del contesto scolastico**. L'obiettivo principale della mia ricerca è stato quello di sviluppare un'applicazione che non solo intrattenga i bambini, ma che offra anche un ambiente educativo e stimolante per il loro sviluppo cognitivo e motorio. La progettazione di un'applicazione per bambini richiede una profonda comprensione delle loro esigenze, delle loro capacità e delle loro preferenze. A tal proposito è stata condotta un'accurata ricerca ed un'attenta analisi dei requisiti necessari alla progettazione.

Sulla base della ricerca condotta e dell'analisi dei requisiti, si stabilito che fosse necessario analizzare numerosi fattori in modo da finalizzare al meglio il lavoro. In primo luogo, è stato necessario considerare l'età dei bambini target, poiché le loro capacità cognitive e motorie variano a seconda delle fasi di sviluppo.

In secondo luogo, è importante considerare gli aspetti educativi dell'applicazione. Mentre l'obiettivo principale è intrattenere i bambini, è altrettanto importante offrire loro un ambiente che stimoli il loro apprendimento e che possa dargli un'idea di "sfida". Questa caratteristica può essere raggiunta mediante dei quiz a punteggio, giochi e attività che garantiscono l'interazione. L'applicazione di fatto è stata sviluppando delle sfide che richiedono problem-solving, abilità linguistiche o abilità matematiche, incoraggiando così lo sviluppo cognitivo dei bambini durante il gioco.

Grazie ai confronti accurati con i colleghi di facoltà specializzate, sono riuscito a ottenere una panoramica esaustiva e dettagliata di un possibile metodo di interfacciamento con il bambino. Questi scambi di idee e conoscenze hanno contribuito a consolidare la mia comprensione dell'approccio più efficace per coinvolgere i bambini.

La progettazione dell'applicazione deve tenere conto delle preferenze dei bambini. Questo ha condotto alla creazione di un'interfaccia intuitiva mediante degli strumenti software appositi, che rispecchi i loro interessi e che sia facilmente accessibile.

È essenziale, infine garantire un ambiente che possa essere gestito da un bambino in maniera autonoma, restando conformi ai criteri di protezione. Ciò implica la protezione dei dati personali e la gestione di contenuti appropriati per l'età deve essere

fondamentale.

In sintesi, la progettazione di un'applicazione per bambini che offra un ambiente di apprendimento divertente ed educativo richiede un'attenta considerazione delle loro esigenze, delle loro capacità, delle loro preferenze e della sicurezza. Attraverso una ricerca approfondita e un'analisi dei requisiti accurata, è possibile creare un'applicazione che non solo intrattenga i bambini, ma che favorisca anche il loro sviluppo cognitivo e motorio in modo efficace e sicuro.

L'applicazione prevede certamente buone prospettive per il futuro, come l'espansione degli argomenti disponibili, oppure l'ampliamento dei minigiochi con altre challenges. Inoltre, integrare nuove tecnologie emergenti, come la realtà virtuale o la realtà aumentata, per offrire un'esperienza ancora più coinvolgente e immersiva. L'uso di tali tecnologie potrebbero permettere ai bambini di interagire con gli oggetti virtuali e di esplorare ambienti simulati, aprendo così nuove possibilità di apprendimento ed esplorazione.

Infine potrebbe essere interessante come prospettiva implementare algoritmi di machine learning, che possano essere utili al monitoraggio e alla valutazione del bambino. Questo consentirebbe oltre che all'applicazione di capire quanto il bambino sia progredito, proponendogli quesiti sempre più complicati, anche di avere statistiche utili ai genitori o agli educatori.

BIBLIOGRAFIA E SITOGRAFIA

- [1] *Apple M1*. URL: <https://www.apple.com/it/newsroom/2020/11/apple-unleashes-m1/>.
- [2] *AppStorage*. URL: <https://developer.apple.com/documentation/swiftUI/AppStorage>.
- [3] *Challenge Based Learning*. URL: https://scuoladigitale.istruzione.it/scenari_cpt/challenge-based-learning/.
- [4] *Differenze tra Front-end e Back-end Developer*. URL: <https://reteinformaticalavoro.it/blog/differenze-tra-front-end-e-back-end-developer/>.
- [5] *Documentazione GameplayKit*. URL: <https://developer.apple.com/documentation/gameplaykit/>.
- [6] *Documentazione Swift*. URL: <https://developer.apple.com/documentation/swift/>.
- [7] *Documentazione SwiftUI*. URL: <https://developer.apple.com/documentation/swiftUI/>.
- [8] *Documentazione UIKit*. URL: <https://developer.apple.com/documentation/uikit/>.
- [9] *Documentazione Xcode*. URL: <https://developer.apple.com/documentation/xcode/>.
- [10] *Indicazioni nazionali e nuovi scenari*. URL: <https://www.miur.gov.it/documents/20182/0/Indicazioni+nazionali+e+nuovi+scenari/>.
- [11] *iPhone 13*. URL: <https://www.apple.com/it/shop/buy-iphone/iphone-13>.
- [12] *MacBook Pro*. URL: <https://www.apple.com/it/shop/buy-mac/macbook-pro/13-pollici-grigio-siderale-chip-apple-m2-con-cpu-8-core-e-gpu-10-core-256gb>.
- [13] *Materiale di riferimento Apple Foundation Program (Ed.2023)*. 2023.
- [14] *Novità in Photoshop*. URL: <https://helpx.adobe.com/it/photoshop/using/whats-new.html>.
- [15] *Sketch*. URL: <https://www.sketch.com/#mac-app>.
- [16] Ian Sommeriville. *Ingegneria del software 10/Ed.* Pearson, 2023.
- [17] *Supporto Apple*. URL: <https://developer.apple.com/support/>.

ELENCO DELLE FIGURE

1	Schema Waterfall di progettazione di un software	5
2	Schermata principale dell'app EduGames	6
1.1	MacBook Pro M1 (2020)	9
1.2	Schema architetturale Apple Silicon M1	10
1.3	iPhone 13 Product Red	11
1.4	interfaccia di XCode	14
1.5	interfaccia generica di Sketch	15
1.6	Sviluppo iniziale dell' interfaccia di Edugames	16
1.7	Sviluppo dell'Icona	17
2.1	CBL circle	19
2.2	Passi della prima fase della CBL	21
2.3	Passi della seconda fase della CBL	23
2.4	Fase finale approccio CBL	25
2.5	Modello di Canvas CBL	25
3.1	Divisione in classi in XCode	28
3.2	Schermata principale dell'app EduGames	29
3.3	HorizontalDatePicker	30
3.4	Barra di progresso	32
3.5	Interfaccia del Quiz	34
3.6	Codice del costruttore	34
3.7	Frame gamba Dx	35
3.8	Fasi Iniziali	36
3.9	Fasi di gioco	36
3.10	Schema delle classi del gioco e parte del codice	37

RINGRAZIAMENTI

A chiusura dell'elaborato ci tengo, a menzionare le persone senza le quali questo percorso non sarebbe stato possibile.

Un grazie doveroso e sincero al mio relatore, la Professoressa Carmen De Maio, per essere stata sempre disponibile per eventuali consigli e chiarimenti durante lo sviluppo dell'applicazione e la stesura della tesi.

Un grazie infinito a mio nonno Salvatore, con la speranza che da lassù possa vedere terminare questo percorso. La sua guida e il suo sostegno costante mi mancano profondamente, ma so che è con me, continuando a chiedermi quando dovrò sostenere il prossimo esame.

Un grazie infinito a mio nonno Aniello, che non ha potuto vedermi crescere, ma che fin da bambino ha visto in me qualcosa di speciale.

Ringrazio i miei genitori, fonte di ispirazione e motivazione. Nei momenti di difficoltà hanno sempre saputo come sostenermi e spingermi a rialzarmi sempre più forte. Gli sono grato per tutto ciò che hanno fatto per me e per avermi insegnato cosa significa avere dei valori.

Ringrazio mia sorella Mirella, sempre presente e sempre attenta ai miei discorsi e alle mie richieste. Sarai un medico eccellente.

Ringrazio le mie nonne, i miei zii, i miei cugini, da sempre presenti nella mia vita e sempre pronti ad aiutarmi nei momenti di difficoltà.

Ringrazio particolarmente il mio collega ma soprattutto amico Maurizio, con cui oltre ad aver condiviso le innumerevoli ansie pre-esame, ho trascorso tante ore di studio e di viaggio.

Un ringraziamento doveroso ai miei docenti del Liceo Albertini di Nola. Al professore Michele Miele, uno dei primi a aver creduto sin dall'inizio in me e che mi ha spinto ancor di più ad appassionarmi al mondo dell'ingegneria e dell'informatica. Alla professoressa Annamaria Renzullo, che mi ha spinto ad amare la matematica ogni giorno di più. Non posso dimenticare di menzionare i professori Roberto D'Aniello, Michele Moccia, Sabatina Barbarino, Giovanna Napolitano, Anna Bosone ed Elisa Almanzo. Ognuno di loro ha contribuito alla mia crescita accademica e personale, spronandomi a dare sempre il massimo e a perseguire i miei obiettivi con determinazione.

Ringrazio Daniel, mio fratello, la persona di cui più mi fido.

Ringrazio i miei amici, fonte di divertimento e di svago nei momenti di bisogno. In particolare Felice, Pio, Bruscino e il gruppo stobuon.

Ringrazio UOMO (Felice), per tutti gli allenamenti e le sedute di richiamo divino che ci hanno permesso di cancellare i momenti di stress.

Ringrazio Antonio Panico, per avermi sopportato e supportato al bar nei periodi in cui volevo solo stare fuori casa.

Ringrazio il mio amico Raffaele, per essermi stato accanto nei momenti di difficoltà sapendo sempre quali consigli darmi.

Per ultima, non per importanza, ringrazio dal profondo del cuore Lucia. Sei ciò che mi motiva ad essere un uomo migliore, ciò che mi rende sempre più forte e che mi infiamma l'anima. Quel giorno nei tuoi occhi risplendeva un sorriso tale che io credevo di toccare con i miei un limite mai raggiunto prima.